

Projet SM604 – Parties 1 et 2 : MNIST et CIFAR-10

Mathématiques pour le Machine Learning, EFREI Paris, 2025-2026

Sabrina El Hassani, Aude Labat, Thomas Duriaud, Paul Fontaine, Evan Ladeira

Parties 1 et 2 réalisées from scratch (Python/NumPy). CNN partie 2 : PyTorch (option B).

Code source : <https://github.com/ElhSabrina/projet-ML-SM604>

1 Préparation des données

Les 70 000 images (28×28 pixels) sont représentées par des vecteurs $\vec{x} \in \mathbb{R}^{784}$ et normalisées en divisant par 255. Sans cette étape, les scores $o_k = \sum_j a_{k,j} x_j + b_k$ atteignent plusieurs milliers, rendant e^{o_k} numériquement infini dans le softmax et faisant exploser les gradients lors de la descente. Le jeu est séparé en 60 000 images d'entraînement et 10 000 de test. Les étiquettes sont encodées en vecteurs one-hot $y_i^{(k)} \in \{0, 1\}$ avec $\sum_k y_i^{(k)} = 1$, nécessaires pour la cross-entropy.

2 Modèle linéaire

Pour chaque image, on calcule $O = XA^T + b$, où $A \in \mathbb{R}^{10 \times 784}$ (la transposée est nécessaire car $X \in \mathbb{R}^{n \times 784}$ et on souhaite $O \in \mathbb{R}^{n \times 10}$: $(n \times 784) \cdot (784 \times 10) = (n \times 10)$), $b \in \mathbb{R}^{10}$, puis $P = \text{softmax}(O)$ et $\hat{y} = \arg \max_k P_k$ (7850 paramètres). La cross-entropy est choisie car, contrairement à la MSE, son gradient combiné au softmax ne fait pas intervenir la dérivée de l'activation, évitant la saturation du gradient lors de la descente.

Calcul du gradient. On développe $\log P_{i,k} = o_{i,k} - \log \sum_r e^{o_{i,r}}$ et on substitue dans la loss $L_i = -\sum_k y_i^{(k)} \log P_{i,k}$:

$$L_i = -\sum_k y_i^{(k)} o_{i,k} + \log \left(\sum_r e^{o_{i,r}} \right) \underbrace{\sum_k y_i^{(k)}}_{=1} = -\sum_k y_i^{(k)} o_{i,k} + \log \left(\sum_r e^{o_{i,r}} \right).$$

La dérivée par rapport à $o_{i,k}$ (règle de la chaîne u'/u sur le terme logarithmique) :

$$\frac{\partial L_i}{\partial o_{i,k}} = -y_i^{(k)} + \frac{e^{o_{i,k}}}{\sum_r e^{o_{i,r}}} = P_{i,k} - y_i^{(k)}.$$

On pose $\delta = P - Y \in \mathbb{R}^{n \times 10}$. Comme $\partial o_{i,k} / \partial a_{k,j} = x_{i,j}$ et $\partial o_{i,k} / \partial b_k = 1$, la règle de la chaîne $\frac{\partial L}{\partial \theta} = \frac{\partial L}{\partial o} \cdot \frac{\partial o}{\partial \theta}$ donne :

$$\nabla_A L = \frac{1}{n} (P - Y)^T X \in \mathbb{R}^{10 \times 784}$$

$$\nabla_b L = \frac{1}{n} \sum_{\text{lignes}} (P - Y) \in \mathbb{R}^{10}.$$

Mise à jour : $A \leftarrow A - \eta \nabla_A L$, $b \leftarrow b - \eta \nabla_b L$.

Entraînement. On utilise des mini-batches de 256 images plutôt que le gradient total sur les 60 000 : chaque mini-batch produit une mise à jour de A , soit 234 corrections par epoch au lieu d'une seule, ce qui accélère significativement la convergence. L'arrêt se fait sur $|\Delta L| < 10^{-4}$. Après comparaison de $\eta \in \{0,01; 0,1\}$ (cf. Annexe A), $\eta = 0,1$ est retenu : convergence 4,7× plus rapide pour la même accuracy finale. Le modèle linéaire étant convexe, les deux valeurs convergent vers le même minimum global ; seule la vitesse diffère.

3 Modèles avec couches cachées

Architectures. $H = 1$: couche cachée de $p_1 = 128$ neurones, $A^1 \in \mathbb{R}^{128 \times 784}$, $A^2 \in \mathbb{R}^{10 \times 128}$, 101 770 paramètres. $H = 2$: deux couches $p_1 = 128$, $p_2 = 64$, $A^3 \in \mathbb{R}^{10 \times 64}$, 109 386 paramètres. $p_1 = 128$ est une puissance de 2, ce qui est efficace en termes de calcul matriciel ; c'est aussi suffisamment grand pour capturer des représentations intermédiaires sans alourdir l'entraînement. $p_2 = 64$ réduit la dimension avant la sortie tout en restant dans le même ordre de grandeur. ReLU est choisie comme activation car sa dérivée vaut 1 pour $t > 0$ (contre $\sigma(t)(1 - \sigma(t)) \leq 0,25$ pour sigmoid), ce qui évite que le gradient s'atténue exponentiellement à travers les couches.

Rétropropagation $H = 1$. Passage avant : $o^1 = X(A^1)^T + b^1$, $z^1 = \text{ReLU}(o^1)$, $o^2 = z^1(A^2)^T + b^2$, $P = \text{softmax}(o^2)$. Couche de sortie — règle de la chaîne avec $\partial o_{i,k}^2 / \partial a_{k,q}^2 = z_{i,q}^1$:

$$\delta^2 = P - Y, \quad \nabla_{A^2} L = \frac{1}{n} (\delta^2)^T z^1 \in \mathbb{R}^{10 \times 128}, \quad \nabla_{b^2} L = \frac{1}{n} \sum_{\text{lignes}} \delta^2 \in \mathbb{R}^{10}.$$

z^1 influence L via o^2 , donc $\partial L / \partial z^1 = \delta^2 A^2 \in \mathbb{R}^{n \times 128}$. Comme $z^1 = \text{ReLU}(o^1)$ avec $\text{ReLU}'(t) = \mathbf{1}[t > 0]$, la règle de la chaîne donne :

$$\delta^1 = (\delta^2 A^2) \times \text{ReLU}'(o^1) \in \mathbb{R}^{n \times 128},$$

où $\times$ est le produit terme à terme. Couche cachée avec $\partial o_{i,q}^1 / \partial a_{q,j}^1 = x_{i,j}$:

$$\boxed{\nabla_{A^1} L = \frac{1}{n} (\delta^1)^T X \in \mathbb{R}^{128 \times 784}}, \quad \boxed{\nabla_{b^1} L = \frac{1}{n} \sum_{\text{lignes}} \delta^1 \in \mathbb{R}^{128}}.$$

Rétropropagation $H = 2$. Passage avant : $z^2 = \text{ReLU}(z^1 (A^2)^T + b^2)$, $P = \text{softmax}(z^2 (A^3)^T + b^3)$. On pose $\delta^3 = P - Y$ et on applique le même schéma :

$$\boxed{\nabla_{A^3} L = \frac{1}{n} (\delta^3)^T z^2}, \quad \delta^2 = (\delta^3 A^3) \times \text{ReLU}'(o^2), \quad \boxed{\nabla_{A^2} L = \frac{1}{n} (\delta^2)^T z^1},$$

$$\delta^1 = (\delta^2 A^2) \times \text{ReLU}'(o^1), \quad \boxed{\nabla_{A^1} L = \frac{1}{n} (\delta^1)^T X}.$$

Les biais se mettent à jour comme pour $H = 1$. Ce schéma correspond à la Proposition 5.2 du cours.

Résultats.

Modèle	Paramètres	Erreur train	Erreur test	Accuracy	Écart
Linéaire	7 850	7,50 %	7,80 %	92,20 %	0,30 %
$H = 1$ ($p_1 = 128$)	101 770	1,07 %	2,39 %	97,61 %	1,32 %
$H = 2$ ($p_1 = 128, p_2 = 64$)	109 386	0,33 %	2,43 %	97,57 %	2,10 %

Table 1: Résultats ($\eta = 0,1$, mini-batches 256, cf. Annexe B).

$H = 1$ gagne 5,41 points sur le test par rapport au modèle linéaire. $H = 2$ présente de l'overfitting (écart 2,10 %) : l'erreur train descend à 0,33 % tandis que l'erreur test stagne à 2,43 %. Nous avons testé $p_2 = 32$ (cf. Annexe G) : l'écart reste à 2,25 %, car A^1 concentre 100 352 des 109 386 paramètres. Nous avons également testé un arrêt anticipé sur l'écart train/test : le résultat est identique à la convergence naturelle. On conclut que l'overfitting de $H = 2$ provient du volume de paramètres de la première couche, non de p_2 .

4 Analyse des erreurs et discussion

La matrice de confusion (cf. Annexe C) révèle les confusions principales : 5→3 (14 fois), 4→9 (12 fois), 8→3 (10 fois). La projection PCA (cf. Annexe D) confirme que les classes 3, 5, 6 et 8 se superposent au centre du nuage (17,5 % de variance expliquée), tandis que 0 et 1 restent bien séparés. Ces chiffres partagent des formes proches que le modèle ne peut distinguer en traitant les 784 valeurs du vecteur $\vec{x}$ indépendamment, sans information de voisinage spatial. Cela explique également les erreurs à haute confiance (un 2 prédit comme 7 à 92 %, cf. Annexe E).

Transition. MNIST bénéficie de conditions idéales (fond uniforme, chiffres centrés, pas de rotation) permettant d'atteindre 97,6 % sans exploiter la structure spatiale. Sur des images naturelles (CIFAR-10), la relation entre pixels voisins devient indispensable : les filtres convolutifs de la partie 2 analysent des zones 3×3 pour détecter contours et textures, là où ce modèle est aveugle.

Partie 2 : CIFAR-10

5 Données et préparation

CIFAR-10 contient 60 000 images couleur 32×32 pixels réparties en 10 classes (avion, automobile, oiseau, chat, cerf, chien, grenouille, cheval, bateau, camion), avec 50 000 images d'entraînement et 10 000 de test (cf. Annexe H). Le dataset est parfaitement équilibré : 5 000 images par classe (10 % chacune).

Deux représentations sont testées. **Grayscale** : $x_j = 0,299 R_j + 0,587 G_j + 0,114 B_j$, donnant $\vec{x} \in \mathbb{R}^{1024}$ (cf. Annexe I). **RGB** : les 3 canaux aplatis bout à bout, donnant $\vec{x} \in \mathbb{R}^{3072}$. Dans les deux cas, les pixels sont normalisés entre 0 et 1.

6 Travail préliminaire : architectures denses

On applique les trois architectures de la Partie 1 en adaptant uniquement $n_{\text{input}} \in \{1024, 3072\}$. Toutes les fonctions sont identiques et centralisées dans `utils.py`.

Modèle	Entrée	Paramètres	Erreur train	Erreur test	Écart
Linéaire	Gray 1024	10 250	74,40 %	76,13 %	1,73 %
$H = 1$ (128)	Gray 1024	132 490	66,41 %	68,87 %	2,46 %
$H = 2$ (128,64)	Gray 1024	140 106	62,40 %	65,64 %	3,24 %
Linéaire	RGB 3072	30 730	79,33 %	80,01 %	0,68 %
$H = 1$ (128)	RGB 3072	394 634	53,90 %	59,34 %	5,44 %
$H = 2$ (128,64)	RGB 3072	402 250	47,21 %	52,14 %	4,93 %

Table 2: Résultats des architectures denses sur CIFAR-10 (cf. Annexe J).

Observations. Le modèle linéaire RGB (80 %) est plus mauvais que le linéaire grayscale (76 %) : tripler la dimension d’entrée sans couche cachée multiplie le bruit à apprendre sans ajouter de capacité d’abstraction. Les couches cachées exploitent mieux la couleur : $H = 2$ RGB (52 %) surpasse son équivalent grayscale (66 %). Les courbes linéaires oscillent sans converger sur 50 epochs (cf. Annexe J) : CIFAR-10 n’est pas linéairement séparable dans l’espace des pixels. Ces résultats confirment la limite fondamentale des architectures denses : elles traitent les pixels indépendamment et ne généralisent pas quand l’objet bouge.

7 Filtres de convolution

On implémente from scratch la convolution 2D avec zero-padding : $m'_{u,v} = \sum_{u'=1}^3 \sum_{v'=1}^3 K_{u',v'} m_{u+u'-2, v+v'-2} + l$. On applique les 6 filtres du sujet ($l = 0$) sur une image de chat (cf. Annexe K). L’observation révèle que K1 lisse l’image (moyennage des voisins) ; K2 accentue les zones de fort contraste ; K3 et K4 font apparaître les transitions horizontales ; K5 isole les contours de l’objet (filtre de Sobel) ; K6 accentue les transitions diagonales. Ces filtres illustrent ce qu’un CNN apprend automatiquement lors de l’entraînement.

8 CNN (Option B : PyTorch)

Architecture. On suit l’architecture du sujet (section 2.5) : 4 couches de convolution (64 filtres, noyau 3×3 , padding=1, ReLU), 2 max-poolings 2×2 , 1 couche dense finale (cf. Annexe L). Après deux poolings, une image 32×32 devient 8×8 , soit $8 \times 8 \times 64 = 4096$ valeurs envoyées à la couche dense. L’optimiseur Adam adapte le learning rate par paramètre, plus robuste qu’une descente classique sur une architecture profonde.

Nombre de paramètres. Le CNN contient 153 546 paramètres, moins que $H = 2$ RGB (402 250), pour un résultat bien meilleur (24,53 % vs 52,14 %). L’efficacité vient du partage des poids : un filtre 3×3 détecte le même motif quelle que soit sa position. Les premières couches apprennent des bords comparables au filtre K5, les couches profondes combinent ces bords en textures puis en parties d’objets.

Résultats et overfitting.

Modèle	Entrée	Paramètres	Erreur train	Erreur test	Écart
$H = 2$ (128,64)	RGB 3072	402 250	47,21 %	52,14 %	4,93 %
CNN PyTorch	RGB 32x32	153 546	3,30 %	24,53 %	21,23 %
Conv Deep Belief Nets (2010)				21,10 %	
Maxout Networks (2013)				9,38 %	
Vision Transformer (2021)				0,50 %	

Table 3: CNN vs meilleur modèle dense et littérature (cf. Annexe M).

Le CNN atteint 3,30 % d’erreur train mais 24,53 % en test (écart 21,23 %). La loss test atteint son minimum à l’epoch 6 (23,02 %) puis remonte alors que la loss train continue de baisser (cf. Annexe M) : le modèle mémorise les données plutôt que de généraliser. Trois facteurs expliquent cet overfitting : le nombre d’epochs fixé à 20 est trop élevé (un early stopping sur la loss test aurait arrêté l’entraînement à l’epoch 6) ; le modèle a une capacité suffisante pour mémoriser 50 000 images ; aucune technique de régularisation n’est présente. Nous avons entendu parler de techniques à approfondir pour améliorer les performances, telles que le dropout, le batch normalization et la data augmentation ; ces pistes constituent des perspectives d’amélioration hors du périmètre de ce projet.

Notre CNN se rapproche du premier article CNN de référence (21,10 %, 2010), ce qui est cohérent avec une architecture simple sans régularisation. Le gain de 27,61 points par rapport au meilleur modèle dense confirme que la convolution est indispensable pour les images naturelles.

9 Partie 3 : Application au diagnostic médical

Le dataset CBIS-DDSM n'a pas pu être traité dans ce dépôt. Les images d'origine atteignent 4000×3000 pixels, nécessitent un redimensionnement vers 128×128 , un matching avec le CSV d'étiquettes (bénin/malin), et un entraînement CNN de plusieurs heures sur CPU. Cette partie sera intégralement traitée dans le dépôt final (semaine 24).

Annexe A : Comparaison des learning rates

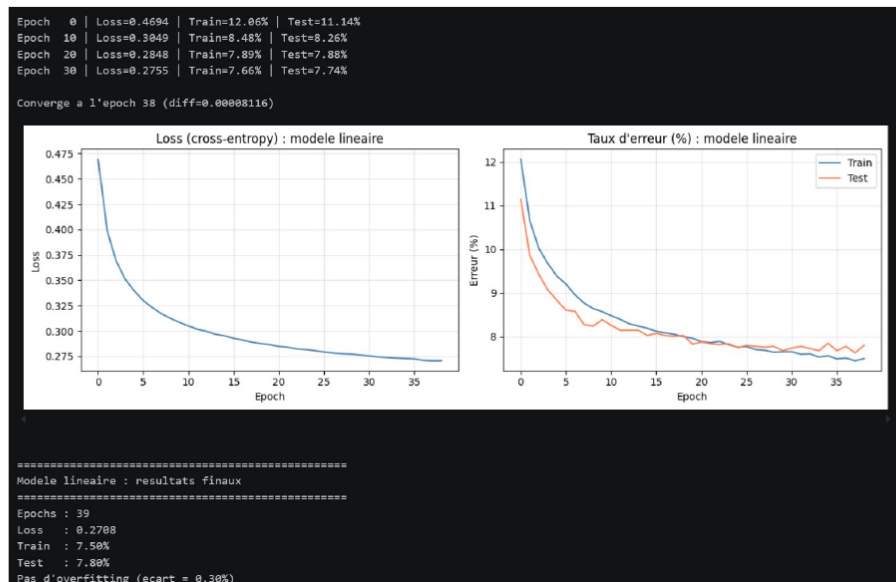
η	Epochs	Erreur train	Erreur test	Verdict
0,1	39	7,50 %	7,80 %	Retenu
0,01	184	8,03 %	8,00 %	4,7 $\times$ plus lent

Annexe B : Courbes d'apprentissage (Partie 1)

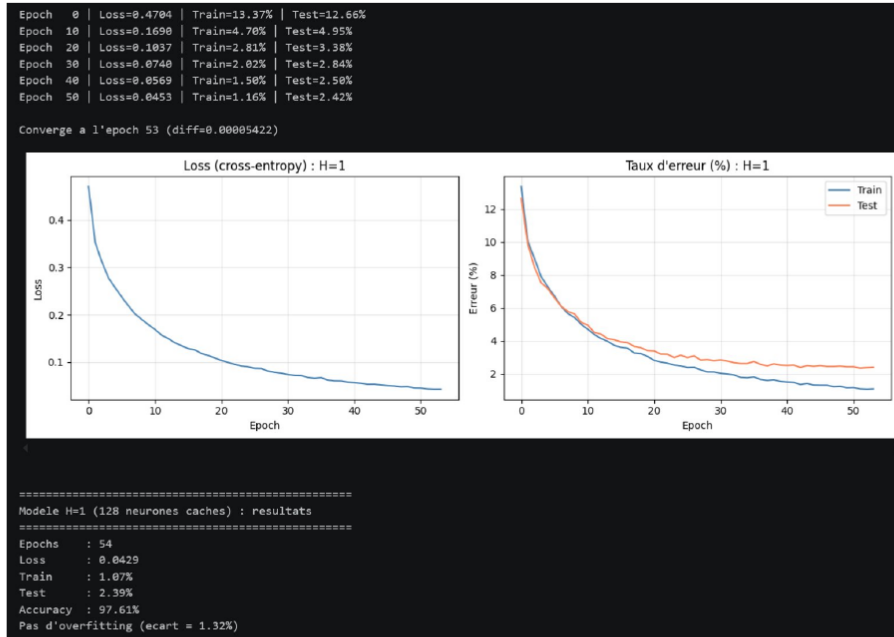
Annexe A : Comparaison des learning rates

η	Epochs	Erreur train	Erreur test	Verdict
0,1	39	7,50 %	7,80 %	Retenu
0,01	184	8,03 %	8,00 %	4,7 $\times$ plus lent

Annexe B : Courbes d'apprentissage



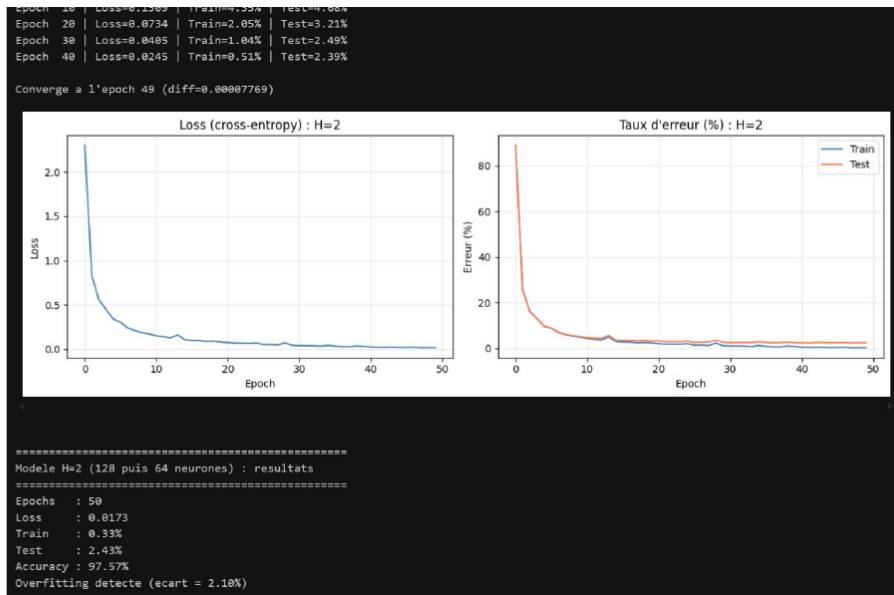
Modèle linéaire : loss et taux d'erreur train/test.



Modèle $H = 1$ ($p_1 = 128$) : loss et taux d'erreur train/test.

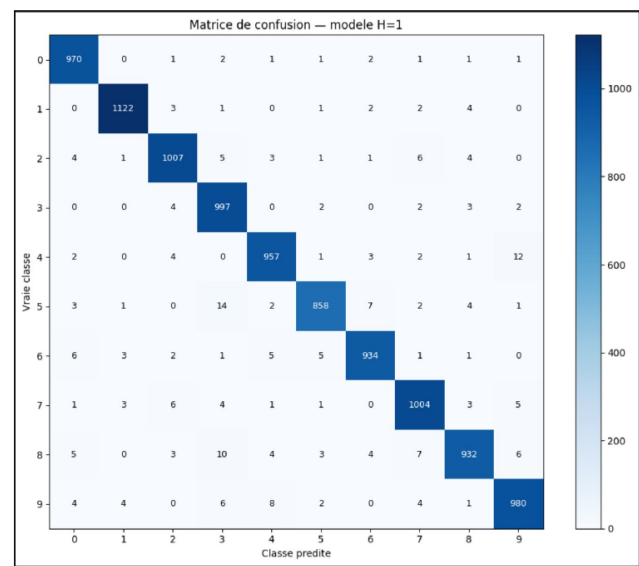
Epoch	0	Loss=2.2971	Train=88.76%	Test=88.65%
Epoch	10	Loss=0.1509	Train=4.35%	Test=4.68%
Epoch	20	Loss=0.0734	Train=2.05%	Test=3.21%
Epoch	30	Loss=0.0405	Train=1.04%	Test=2.49%

Modèle $H = 1$ ($p_1 = 128$) : loss et taux d'erreur train/test.

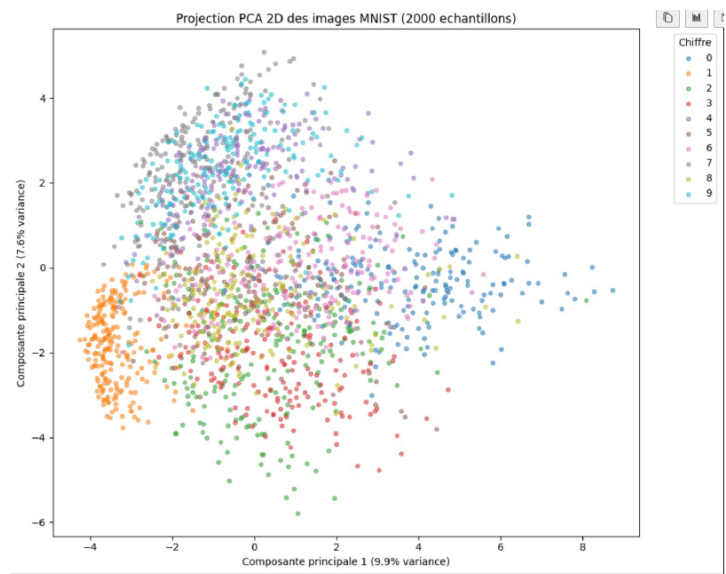


Annexe C : Matrice de confusion (modèle $H = 1$)

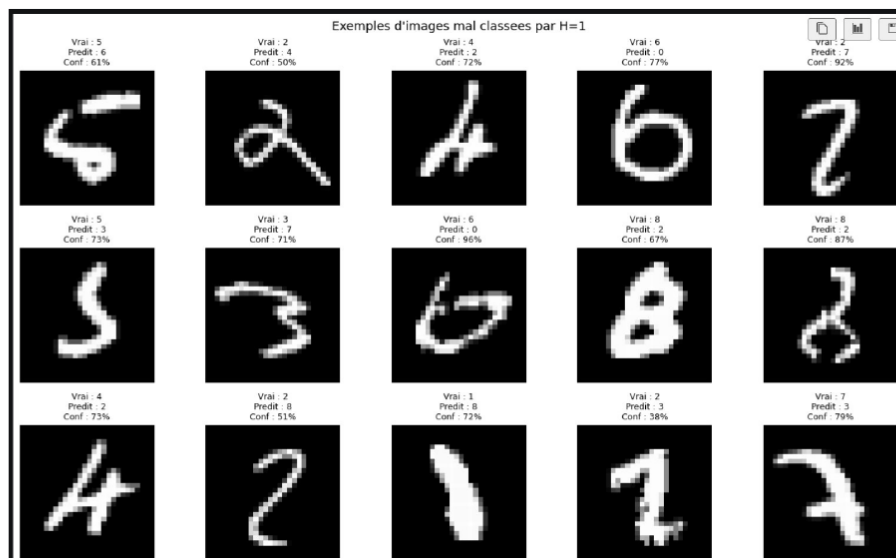
Annexe C : Matrice de confusion (modèle $H = 1$)



Annexe D : Projection PCA 2D (2 000 images de test)



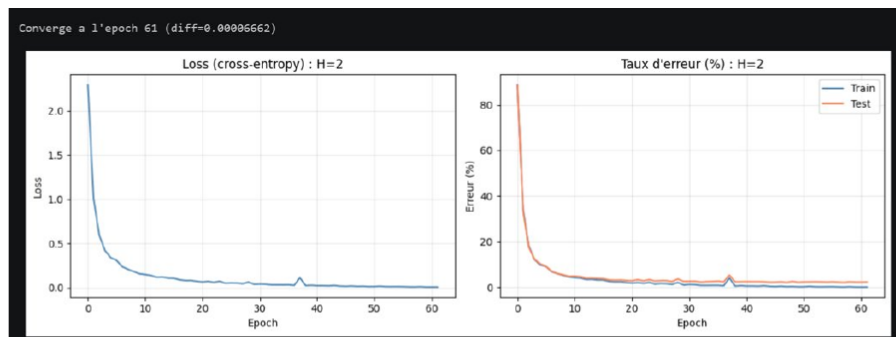
Annexe E : Exemples d'images mal classées (modèle $H = 1$)



Annexe F : Code source

Implémentation from scratch en Python/NumPy disponible dans `MML_Partie1_MNIST.ipynb` et `MML_Partie2_CIFAR10.ipynb`. Les fonctions communes sont centralisées dans `utils.py`.

Annexe G : Test avec $p_2 = 32$ (modèle $H = 2$)



Modèle $H = 2$ avec $p_2 = 32$ (102 282 paramètres) : l'écart train/test reste à 2,25 %, confirmant que l'overfitting provient de A^1 et non de p_2 .

Modèle $H = 2$ avec $p_2 = 32$ (102 282 paramètres) : l'écart train/test reste à 2,25 %, confirmant que l'overfitting provient de A^1 et non de p_2 .

Annexe H : Echantillon CIFAR-10



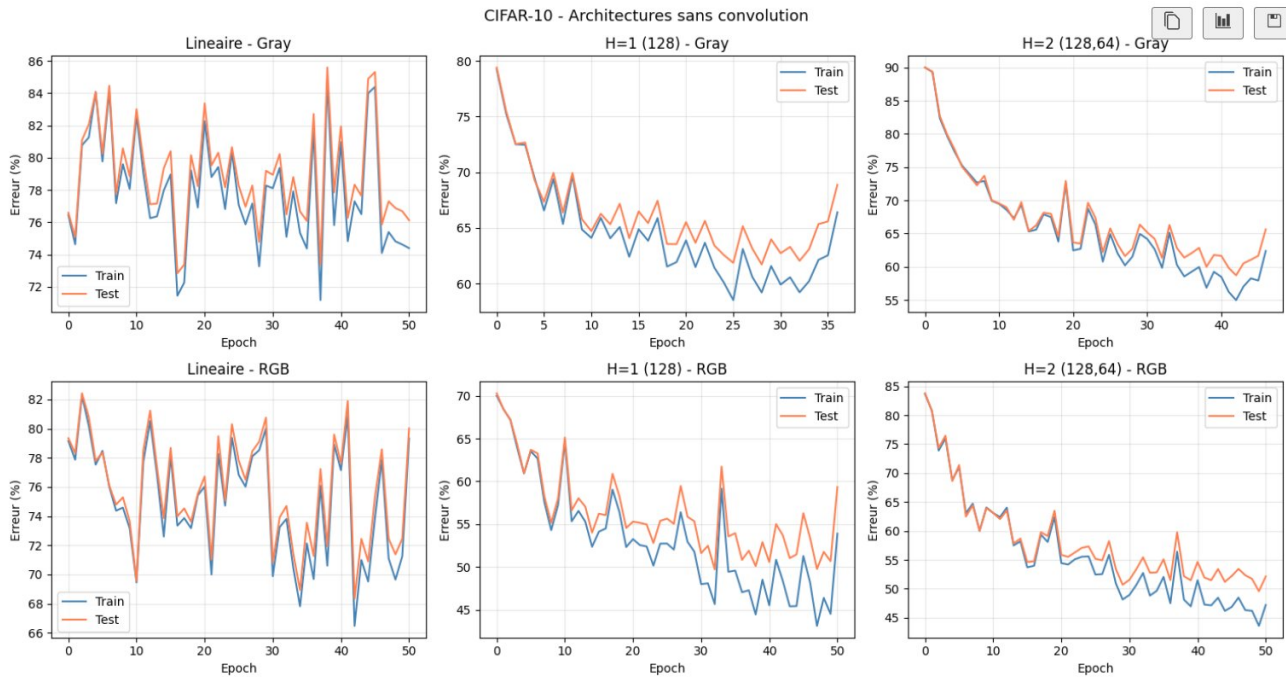
10 images aléatoires de CIFAR-10. 5 000 images par classe exactement – dataset parfaitement équilibré.

Annexe I : Conversion RGB vers niveaux de gris



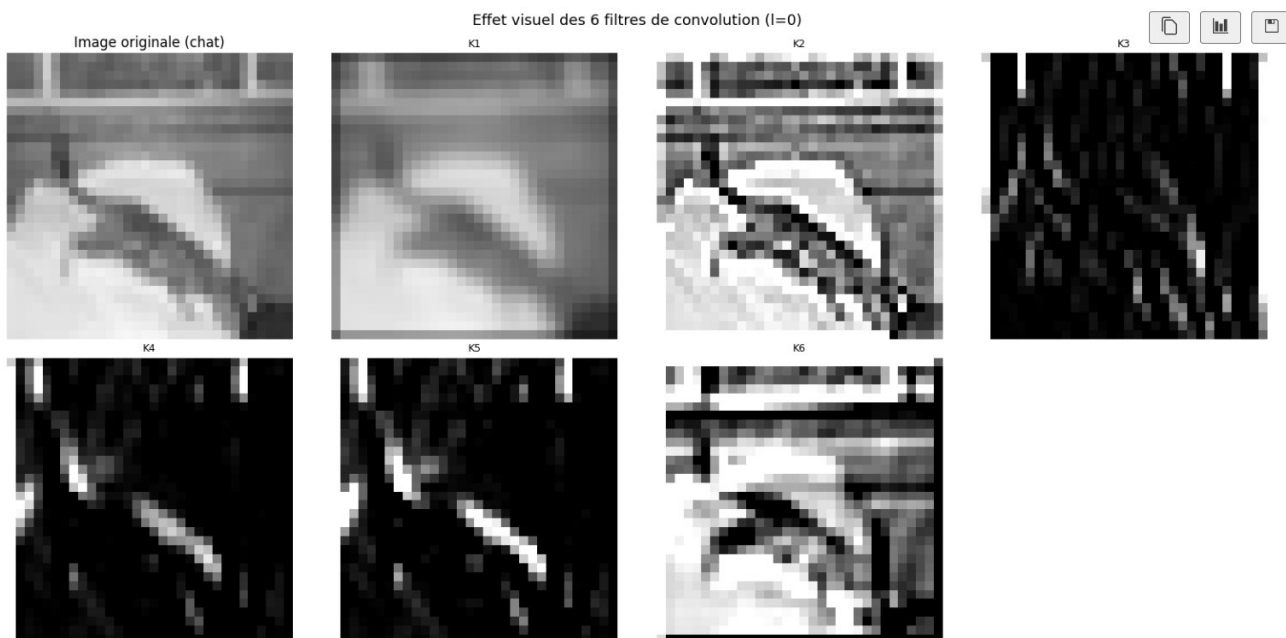
Même image avant (RGB) et après conversion grayscale par $x_j = 0,299 R_j + 0,587 G_j + 0,114 B_j$.

Annexe J : Courbes d'apprentissage – architectures denses CIFAR-10



Taux d'erreur train/test pour les 6 modèles denses. Les courbes linéaires oscillent sans converger : CIFAR-10 n'est pas linéairement séparable.

Annexe K : Effet visuel des 6 filtres de convolution



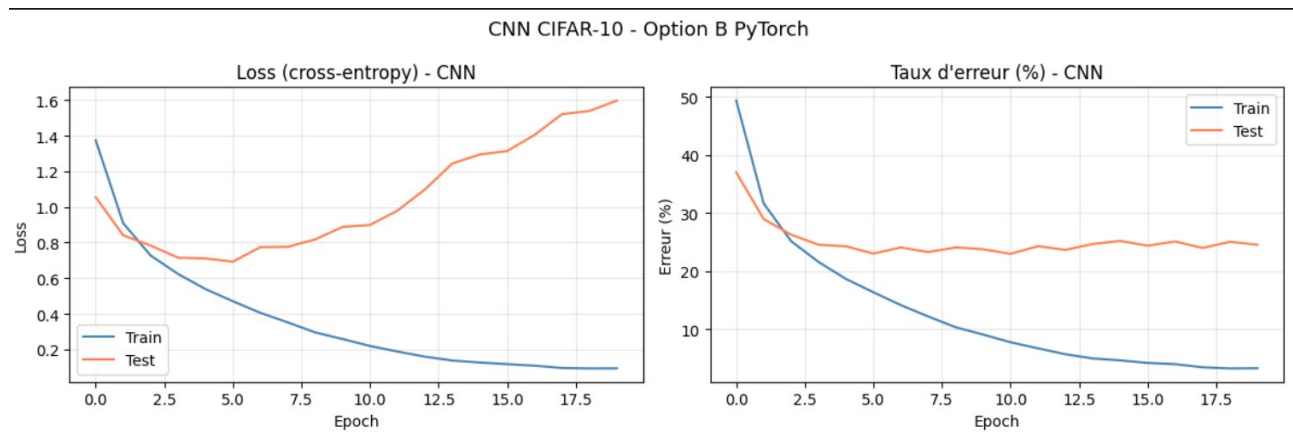
Filtres K1 à K6 ($l = 0$) appliqués sur une image de chat 32×32 . K1 : flou. K2 : contrastes. K3/K4 : transitions horizontales. K5 : contours. K6 : diagonal.

Annexe L : Architecture CNN

Couche	Opération	Sortie
Conv 1	Conv2d(3→64, 3×3, pad=1) + ReLU	(64, 32, 32)
Conv 2	Conv2d(64→64, 3×3, pad=1) + ReLU	(64, 32, 32)
Pool 1	MaxPool2d(2×2)	(64, 16, 16)
Conv 3	Conv2d(64→64, 3×3, pad=1) + ReLU	(64, 16, 16)
Pool 2	MaxPool2d(2×2)	(64, 8, 8)
Conv 4	Conv2d(64→64, 3×3, pad=1) + ReLU	(64, 8, 8)
Flatten	–	4 096
Dense	Linear(4096→10)	10

Total : 153 546 paramètres entraînables.

Annexe M : Courbes d'apprentissage – CNN PyTorch



Loss et taux d'erreur train/test du CNN sur 20 epochs. La loss test atteint son minimum à l'epoch 6 (23,02 %) puis remonte : overfitting caractéristique d'un CNN sans régularisation.